

Fianza: Uncollateralized Credit for Autonomous Agents

A revenue-underwritten lending protocol on Stellar

Divyanshh Kalra

Kundan Kumar

Version 1.0 • August 2026 • `docs.fianza.space`

Abstract

An autonomous software agent can now earn money. It sells inference, research and data over machine-payable APIs and is paid in stablecoins, on-chain, without a human in the loop. It still cannot borrow. No bank underwrites a process, and collateralized lending requires it to already hold the capital it is trying to raise. The result is a class of economically productive actors that can generate income but cannot finance the inputs that income depends on.

We propose a lending protocol that underwrites an agent's *verifiable trailing revenue* rather than its assets. The central difficulty is not reading revenue—inflows are public—but establishing that revenue represents genuine third-party economic activity rather than an operator paying themselves from wallets they control. We show that this cannot be *decided*, only *priced*, and we give a continuous discount model that converts raw inflows into effective independent revenue by weighting each payer on account age, external counterparty diversity, funding-graph independence and net-flow reciprocity, then penalising concentration. Credit is sized as a tier multiple of that figure, and further restricted by a repayment-history ramp beginning at 15% of the sized limit. Each borrower's lenders occupy an isolated vault, so a default is contained to the parties who chose that borrower. Defaults are triggered permissionlessly, absorbed first by a reserve funded from interest, and otherwise socialised pro-rata through share price. The system is implemented in three Soroban contracts settling in USDC and is live on Stellar mainnet.

1. Introduction

Machine-payable APIs made agents into economic participants. An agent can hold a keypair, receive stablecoin payments for work performed, and pay for its own inputs. Payment is solved. Credit is not.

Consider an agent that reliably earns \$5 for a research task requiring a \$2 data purchase. It is profitable. If its balance is below \$2 at the moment work arrives, it cannot begin, and the profit is lost. Traditional finance cannot serve it: underwriting assumes a legal person, a jurisdiction, and recourse. Collateralized DeFi cannot serve it either—a protocol requiring \$150 of collateral to lend \$100 is useless to a borrower whose defining characteristic is having no capital and a real income stream.

The gap is not liquidity. It is underwriting. What is missing is a way to convert observable income into a credit decision a contract can enforce.

Three properties make this newly tractable. Agent revenue arrives on a public ledger, so it is observable without self-reporting. Agents are programs, so borrowing and repayment can be automated to a degree no human borrower matches. And settlement on a low-fee, fast-finality chain makes loans of a few dollars economically coherent, where elsewhere gas alone would exceed the interest.

Section 3 states the underwriting primitive. Section 4 addresses the adversarial core: distinguishing independent revenue from manufactured revenue. Sections 5 through 9 give the credit, custody and default mechanics. Section 11 analyses economic security, and Section 13 states the limits plainly.

2. Prior approaches and why they do not apply

Collateralized lending. Over-collateralized protocols avoid underwriting entirely: the loan is safe because the collateral exceeds it. This is robust and useless here, since it presupposes the capital the borrower lacks.

Credit scoring from wallet history. Several systems score addresses on transaction history, protocol interaction or asset holdings. These measure *activity*, not *income*, and activity is cheap to manufacture. A score that can be farmed is not an underwriting signal.

Off-chain underwriting with on-chain settlement. Real-world credit protocols underwrite legal entities off-chain and settle on-chain. This works, and does not generalise to agents: there is no entity to underwrite, no jurisdiction, and no recourse.

Reputation and staking. Requiring a bond makes misbehaviour costly but again requires capital up front, reproducing the original problem.

Our approach differs in what it treats as the primitive: not the borrower’s assets, identity or reputation, but the *cash flows arriving at its address*, and the degree to which those flows can be shown to originate from parties the borrower does not control.

3. Revenue as an underwriting primitive

Let an agent hold address a . Over a trailing window we observe the set of stablecoin transfers into a . Each transfer has a payer, an amount and a timestamp, all publicly verifiable. Define raw revenue as

$$R_{\text{raw}} = \sum_{i \in P} r_i \tag{1}$$

where P is the set of distinct payers and r_i the total received from payer i .

R_{raw} is worthless as an underwriting input. It is trivially inflated: an operator creates n addresses, funds them, and cycles capital through them. The entire problem is converting R_{raw} into a figure reflecting only revenue the operator could not have manufactured cheaply.

We additionally admit off-chain revenue proven by zero-knowledge TLS attestation—an agent proving a payment-processor balance without revealing credentials—verified by an on-chain verifier contract. Proven off-chain revenue is weighted at $1.5\times$ relative to raw on-chain revenue, since it carries an external attestation on-chain revenue lacks.

4. The independence problem

4.1 Statement

An underwriter must answer: *is this revenue genuine independent economic activity, or the operator paying itself?*

This admits no proof. Nothing distinguishes, at the ledger level, a payment from a real customer from a payment by the operator’s second wallet. Both are valid transfers of real value. The distinguishing fact—common control—is not observable.

We therefore restate the goal. We do not attempt to detect fake revenue. We attempt to make manufacturing *counted* revenue more expensive than the credit it unlocks. The protocol is solvent by construction if

$$\text{cost_to_fake}(R) > \text{value_unlocked}(R) \quad (2)$$

Since credit is at most $3\times$ counted revenue and the ramp restricts a cold borrower to 15% of that, an attacker must cycle real capital repeatedly through apparently unrelated wallets to extract a small multiple. Every mechanism below exists to raise the left side of (2).

4.2 Threat model

We assume an adversary who can create unlimited addresses at negligible cost, move their own capital freely between them, wait to age accounts, script plausible activity, and obtain genuine external payment-processor accounts. We assume they *cannot* cheaply obtain other people’s independent economic activity. That single asymmetry is the foundation of the model.

ID	Attack	Mechanism
A1	Self-payment	Agent pays itself from controlled wallets
A2	Fresh-wallet farm	n new addresses with no other history
A3	Circular funding	Agent funds wallets, they pay it back
A4	Concentration	Two or three payers supply nearly all revenue
A5	Synthetic cadence	Scripted, regular payments imitating volume
A6	Off-chain self-charge	Operator charges their own cards
A7	Collusion ring	Distinct real operators cross-pay to inflate each other

4.3 Per-payer weighting

Each payer i receives a weight $w_i \in [0, 1]$, the product of four independent factors. A product, not a sum: any single factor going to zero eliminates that payer’s contribution entirely, which is the correct behaviour when a payer is provably circular.

$$w_i = \alpha_i \cdot \delta_i \cdot \phi_i \cdot \rho_i \quad (3)$$

Age. $\alpha_i = \text{clamp}_{[0,1]} \left(\frac{\text{age}_i}{A} \right)$, where age_i is the payer’s account age and A a maturity threshold. Defeats A2: a wallet created to pay the agent contributes nothing.

External diversity. $\delta_i = \text{clamp}_{[0,1]} \left(\frac{d_i}{D} \right)$, where d_i is the number of distinct counterparties the payer transacts with *excluding the agent and the agent's other payers*, and D a diversity threshold. Defeats A1 and weakens A7: a wallet that only ever deals with this agent is a puppet; one that deals only with the agent and its co-payers is a ring member. Both show zero external out-degree.

Funding independence. $\phi_i \in \{0, 1\}$, zero if the payer's funds trace back to the agent within K hops of the transfer graph, one otherwise. Defeats A3 directly. We perform a breadth-first traversal of funding sources to depth $K = 3$.

Reciprocity. $\rho_i = \text{clamp}_{[0,1]} \left(1 - \frac{p_i}{r_i} \right)$, where p_i is what the agent paid *back* to payer i . Mutual cross-payment nets toward zero, attacking A7: a ring whose members pay each other in both directions produces little counted revenue.

4.4 Concentration

Even with genuine, independent payers, revenue concentrated in one or two counterparties is fragile and easier to fabricate. Two mechanisms apply. First, each payer's contribution is capped at a maximum share θ of raw revenue before weighting:

$$c_i = \min(r_i, \theta \cdot R_{\text{raw}}) \quad (4)$$

Second, we compute a normalised Herfindahl–Hirschman index over capped shares $s_i = c_i / \sum_j c_j$:

$$H = \sum_i s_i^2 \quad (5)$$

and derive a concentration factor that is 1 for well-diversified revenue and decays to 0 as revenue concentrates:

$$\gamma = 1 - \text{clamp}_{[0,1]} \left(\frac{H - H_0}{1 - H_0} \right) \quad (6)$$

where H_0 is a tolerance floor below which no penalty applies. This addresses A4.

4.5 Temporal organicity

Genuine demand is irregular. Scripted payments are not. We compute a factor $\omega \in [0, 1]$ from the dispersion of inter-payment intervals, penalising distributions that are implausibly uniform. This raises the cost of A5 without claiming to defeat a patient adversary who deliberately randomises.

4.6 Aggregation

Effective independent revenue is

$$R_{\text{eff}} = \gamma \cdot \omega \cdot \sum_{i \in P} w_i c_i \quad (7)$$

R_{eff} , never R_{raw} , is what enters scoring. The ratio $R_{\text{eff}}/R_{\text{raw}}$ is published as an independence score, making the discount auditable rather than opaque.

The output is a discount, not a verdict. A payer is not accepted or rejected; it is worth some fraction of its face value. This matters: a binary classifier has a decision boundary an

attacker can probe and cross. A continuous discount means that as an attacker’s construction approaches indistinguishability from real revenue, its cost approaches that of real revenue.

5. Scoring and credit sizing

R_{eff} and the count of independent counterparties map to a composite score $S \in [0, 850]$ via revenue bands plus a diversity bonus of 5 points per independent payer up to 10 payers. On-chain revenue counts toward the score only if the agent clears a minimum counterparty threshold; below it, and absent proven off-chain revenue, the score is capped below the lending threshold.

The score bands into a tier, and the tier fixes both the credit multiple and the base interest rate.

Tier	Score	Limit multiple	Base APR
A	≥ 750	$3.0\times$	6.00%
B	≥ 650	$2.0\times$	8.50%
C	≥ 550	$1.0\times$	12.00%
Unrated	< 550	0	—

The sized limit is

$$L_{\text{base}} = R_{\text{eff}} \times m(\text{tier}) \quad (8)$$

An agent with no revenue scores Unrated and receives a zero limit. This is the correct output, not a failure state.

The multiple is deliberately small. At $3\times$ for the best tier, an attacker must manufacture \$1 of *counted* revenue to unlock at most \$3 of exposure, before the ramp reduces it further.

6. The credit ramp

A score derived from revenue says what an agent *could* support. It says nothing about whether it repays. We therefore separate the sized limit from the enforceable limit.

Let n be on-time repayments and k missed. The ramp factor is

$$f = \text{clamp}_{[0,1]}(f_0 + n \Delta^+ - k \Delta^-) \quad (9)$$

with $f_0 = 0.15$, $\Delta^+ = 0.15$ and $\Delta^- = 0.30$. The enforced limit is

$$L = L_{\text{base}} \cdot f \quad (10)$$

Three consequences. A cold agent, however well-scored, draws at most 15% of its sized limit, bounding what a freshly-fabricated identity can extract. Trust is earned linearly and lost at twice the rate. And a degrading agent manifests as a ramp that stops advancing—an observable leading indicator preceding default rather than following it.

This is the mechanism that makes the system robust to the volatility of its borrowers. An agent’s competence may change without warning: its underlying model may be swapped, its market may move. We do not attempt to predict this. We bound exposure at any moment and let history move the bound.

7. Isolated vaults

Each agent has its own vault. A lender does not deposit into a protocol-wide pool; they deposit against a specific borrower, receiving shares in that vault alone. For a vault with total assets A_v and total shares T , a deposit of x mints

$$\Delta T = \begin{cases} x, & T = 0 \\ x \cdot T/A_v, & \text{otherwise} \end{cases} \quad (11)$$

Share price A_v/T therefore rises with accrued interest and falls with realised loss, and every lender's position tracks the performance of the borrower they selected.

Isolation is a deliberate rejection of capital efficiency in favour of containment. A shared pool is more efficient and spreads yield; it also spreads *loss*, and for a borrower class where high default rates are expected rather than exceptional, that is the wrong trade at this stage. A pooled layer can be added above isolated vaults once loss rates are empirically characterised; it cannot be safely removed later.

8. Interest, reserve and yield

A draw starts a term. Interest accrues linearly on outstanding principal at

$$\text{APR} = \text{APR}_{\text{base}}(\text{tier}) + \pi \cdot u \quad (12)$$

where $u = \text{principal}/\text{assets}$ is vault utilisation and π the maximum utilisation premium (10 percentage points at full utilisation). Scarce liquidity is priced upward, which both compensates lenders and dampens demand before a vault is drained.

Repayment applies to interest before principal. Interest is split: a fraction $\sigma = 20\%$ accrues to a first-loss *reserve*, and the remainder becomes claimable lender *yield*:

$$\text{reserve} += \sigma I, \quad \text{yield} += (1 - \sigma)I \quad (13)$$

The reserve is the mechanism by which performing loans pay for failing ones. It is funded by borrowers, not lenders, and accumulates in the specific vault whose borrower generated it.

9. Default

If principal remains outstanding past the due date, **any** party may call `mark_default`. Permissionlessness matters: the protocol operator is not a required participant in loss realisation, and cannot delay recognition of a bad loan.

On default, with outstanding principal D and reserve V :

$$V_{\text{used}} = \min(D, V), \quad \Lambda = D - V_{\text{used}} \quad (14)$$

The reserve is consumed first and returns to withdrawable liquidity. The remainder Λ is written off as realised loss. Because total shares are unchanged while assets fall by Λ , the loss is socialised pro-rata through share price automatically—no iteration over lenders, no privileged distribution step. Accrued interest is written off, the agent is frozen from

further borrowing, and the miss enters its repayment record, collapsing the ramp by Δ^- and reducing its score at the next underwriting pass.

The loss is bounded by the vault. Other agents' vaults are untouched, by construction rather than by policy.

10. Incentives

Borrowers repay because the ramp is the only path to a larger line, and because default freezes borrowing permanently and is recorded on-chain against the address. For an agent whose business depends on working capital, the option value of future credit exceeds a single stolen draw once the ramp has advanced. The system's weakest moment is a cold agent's first draw, which is precisely what $f_0 = 0.15$ bounds.

Lenders earn tier-priced interest plus a utilisation premium, choose their borrower explicitly, and are protected by a reserve their borrower funded. They are not asked to trust agents in general—only the one they selected.

Attackers face inequality (2). Each mechanism raises cost to fake: aging wallets costs time, building external counterparty diversity costs genuine relationships, avoiding the funding graph costs an unlinked capital source, and defeating concentration costs breadth. None is individually insurmountable; the model's claim is that jointly they cost more than $3f$ per unit of counted revenue.

11. Security analysis

Consider an adversary targeting the simplest complete attack: fabricate revenue, draw the maximum, and abscond.

To be counted at all, each fabricated payer must be aged past A , transact with $\lceil D \rceil$ external counterparties that are not the agent's other payers, and be funded from a source not traceable to the agent within K hops. To avoid the concentration penalty the adversary needs several such payers, since a single payer is capped at θ of raw revenue and drives $H \rightarrow 1$, sending $\gamma \rightarrow 0$.

Suppose they assemble this and achieve $R_{\text{eff}} = R$. Tier A requires a high score, so realistically they land in tier B or C, giving $L_{\text{base}} \leq 2R$. Cold, the ramp restricts them to $0.15 \cdot L_{\text{base}} \leq 0.3R$. To do better they must *repay*—returning capital and paying interest—several times, each cycle advancing the ramp by 15%. Reaching the full limit takes six on-time repayments.

The adversary's maximum theft is therefore bounded by roughly $2R$ after sustained good behaviour, having cycled real capital through independent-looking wallets throughout, and having paid interest that funded the reserve now absorbing part of their default. Whether this inequality holds in practice is an empirical question only mainnet loss data can settle; we state the target rather than claim it is met.

The open weakness. A sophisticated, non-reciprocal collusion ring—several genuinely distinct operators, with real external activity, each paying one direction only—is not distinguishable from real customers at the graph level. Reciprocity nets out mutual rings; external diversity catches closed ones. A ring that is neither mutual nor closed defeats

the current model. We believe this requires either staking, which reintroduces a capital requirement, or global community detection over a longitudinal graph. We consider this unsolved and state it rather than obscure it.

12. Implementation

The protocol runs as three Soroban contracts on Stellar, settling in USDC via its SEP-41 contract interface.

score_registry stores each agent’s published score, tier, revenue basis and repayment record. Publication is signer-authorised; reads are open. It is the single source of truth other contracts consult.

credit_line is a read-only view deriving terms—tier, ramped limit, APR—from a published score, using the same shared policy library the vault enforces. Quoted terms and enforced terms cannot diverge, because they are the same code.

lending_vault holds custody. Deposits, share accounting, borrows, the repayment waterfall, reserve, yield, permissionless default and loss socialisation all live here. It independently re-derives the limit from the registry rather than trusting a caller.

A shared **revenue_math** library holds all policy—banding, multiples, APR, ramp, interest split, utilisation—and is compiled into every contract that needs it. This is the invariant keeping the system coherent: there is one definition of each credit rule.

Underwriting itself is off-chain: indexing transfer events, traversing the funding graph, computing R_{eff} , and producing a signed score. The computation is heavy and iterative, and is not a good fit for on-chain execution. Its *output* is on-chain and its *inputs* are public, so any third party can recompute a published score from ledger data and detect a dishonest underwriter. Trust is minimised by verifiability rather than by execution location.

Stellar is load-bearing here, not incidental. Loans in this system are often under ten dollars. On a chain where a settlement costs more than the interest on such a loan, the product does not exist. Sub-cent fees and roughly five-second finality are what make per-draw on-chain settlement coherent at this size.

13. Limitations

We state these plainly, since a credit protocol that hides its failure modes is not one worth lending through.

A single-signer trust root. Scores are published by one authorised signer today. A compromised signer could publish fraudulent scores. Scores are recomputable from public data, so fraud is detectable, but detection is not prevention. Multi-signature attestation is the necessary next step.

Collusion rings are unsolved, as described in Section 11.

Off-chain revenue is partially trusted. A zkTLS proof establishes that a balance exists, not that it represents independent income. An operator charging their own cards produces

a valid proof (A6). We weight proven off-chain revenue above raw on-chain revenue, which is defensible for authenticity and generous for independence.

Parameters are calibrated on thin data. The thresholds A , D , θ , H_0 and the ramp constants are chosen from reasoning and adversarial testing, not from observed default behaviour at scale, which does not yet exist.

Agent behaviour is not modelled. We deliberately do not attempt to predict whether an agent will keep performing. Our protection is exposure bounding—short terms, small multiples, a ramp—not forecasting. If a genuinely earning agent degrades, the loss is real and lands on that vault’s lenders.

14. Conclusion

We have described a lending protocol that underwrites autonomous agents on verifiable revenue rather than collateral. The contribution is not the vault, the interest curve or the default path—those are standard. It is the treatment of counterparty independence as an economic quantity rather than a classification problem: revenue is discounted continuously by how expensive it would have been to manufacture, and credit is sized against what survives that discount.

We claim no proof of independence. We claim a construction in which faking counted revenue costs more than the credit it unlocks, bounded further by a repayment ramp and contained by per-borrower isolation. Whether the inequality holds under a real adversary with real money is the question the next phase must answer, and it can only be answered on mainnet.

The system is live. The contracts are deployed on Stellar mainnet and testnet, the loop has executed end to end in real USDC, and a default has been triggered and absorbed on-chain exactly as specified here.

References

- [1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
- [2] Coinbase et al. *x402: An HTTP-native payment protocol*. 2025.
- [3] Stellar Development Foundation. *Soroban smart contract platform*.
- [4] Stellar Development Foundation. *SEP-41: Token Interface*.
- [5] Reclaim Protocol. *zkTLS attestations for web data*.
- [6] A. Hirschman. *The Paternity of an Index*. American Economic Review, 1964.
- [7] Fianza. *Sybil and counterparty-independence model*. docs.fianza.space/sybil-model
- [8] Fianza. *Credit engine*. docs.fianza.space/credit-engine

A. Parameters

Symbol	Meaning	Value
K	Funding-graph traversal depth	3 hops
θ	Max single-payer share before weighting	0.40
H_0	HHI tolerance floor	0.15
f_0	Ramp start	15%
Δ^+	Ramp gain per on-time repayment	+15%
Δ^-	Ramp penalty per miss	-30%
σ	Interest share to reserve	20%
π	Max utilisation premium	+10.00%
—	Tier A / B / C multiple	3.0× / 2.0× / 1.0×
—	Tier A / B / C base APR	6.00% / 8.50% / 12.00%
—	Minimum independent counterparties	3
—	Off-chain (zkTLS) revenue weight	1.5×

Age and diversity thresholds A and D are network-calibrated; the values used on testnet are deliberately lower than mainnet targets because the chain and the agents on it are young, and are documented in the Sybil model reference.

B. Worked example

An agent has received \$120 from four payers: \$60, \$30, \$20, \$10.

Capping. $R_{\text{raw}} = 120$, so $\theta R_{\text{raw}} = 48$. The largest payer is capped: $c = (48, 30, 20, 10)$, $\sum c = 108$.

Weighting. Payer 1 is mature and externally diverse, $w_1 = 1$. Payer 2 was funded by the agent two hops back, so $\phi_2 = 0$ and $w_2 = 0$. Payer 3 is fresh at half the age threshold with adequate diversity, $w_3 = 0.5$. Payer 4 is mature but the agent paid \$4 back, so $\rho_4 = 1 - 4/10 = 0.6$ and $w_4 = 0.6$.

$$\sum_i w_i c_i = 48 + 0 + 10 + 6 = 64$$

Concentration. Shares over capped total: (0.4444, 0.2778, 0.1852, 0.0926), giving $H = 0.3176$. With $H_0 = 0.15$:

$$\gamma = 1 - \frac{0.3176 - 0.15}{0.85} = 0.8029$$

Aggregation. Assuming organicity $\omega = 1$:

$$R_{\text{eff}} = 0.8029 \times 64 \approx 51.38$$

The agent presents \$120 of inflows and is underwritten on \$51.38—an independence score of 0.428.

Sizing. Suppose this scores into tier C: $L_{\text{base}} = 51.38 \times 1.0 = 51.38$. With no repayment history, $f = 0.15$, so the enforceable limit is

$$L = 51.38 \times 0.15 \approx \$7.71$$

From \$120 of raw inflows, a cold agent may draw under eight dollars. Six on-time repayments bring the ramp to 100%, at which point the full \$51.38 becomes available. That gap between apparent and extractable credit is the protocol's primary defence.